

## Sujet B

## □ Exercice : type A

On rappelle que le principe de la compression LZW est d'attribuer un code aux préfixes rencontrés lors de la lecture du texte à compresser de façon à disposer d'un code compact si ce code se présente à nouveau. Initialement, seuls les codes de l'alphabet (usuellement les caractères ASCII) sont présents dans la table de codage.

Ici, on s'intéresse à des chaînes de caractères représentant des séquences génétique codés sur l'alphabet  $\{A, C, G, T\}$ , par souci de simplicité on attribue initialement les codes  $A \rightarrow 0$ ,  $C \rightarrow 1$ ,  $G \rightarrow 2$  et  $T \rightarrow 3$ . Et on souhaite compresser la séquence **ATGTGATGTCCT**. Le début de l'algorithme va donc consister à attribuer un nouveau code au premier préfixe non encore codé qui apparaît lors de la lecture du texte. Et donc, ici, on attribue le code 4 au préfixe **AT** et on emmettra le code de **A** c'est à dire 0.

1. Poursuivre le déroulement de cet algorithme en complétant le tableau suivant :

| Position dans le texte | Code émis | Nouveau préfixe ajouté |
|------------------------|-----------|------------------------|
| <u>A</u> TGTGATGTCCT   | 0         | AT $\rightarrow$ 4     |

| Position dans le texte | Code émis | Nouveau préfixe ajouté |
|------------------------|-----------|------------------------|
| <u>A</u> TGTGATGTCCT   | 0         | AT $\rightarrow$ 4     |
| AT <u>G</u> TGATGTCCT  | 3         | TG $\rightarrow$ 5     |
| ATGT <u>G</u> ATGTCCT  | 2         | GT $\rightarrow$ 6     |
| ATGTGAT <u>G</u> TCCT  | 5         | TGA $\rightarrow$ 7    |
| ATGTGATG <u>T</u> CCT  | 4         | ATT $\rightarrow$ 8    |
| ATGTGATGT <u>C</u> CT  | 6         | GTT $\rightarrow$ 9    |
| ATGTGATGTCT <u>C</u>   | 1         | CC $\rightarrow$ 10    |
| ATGTGATGTCTC <u>T</u>  | 1         | CT $\rightarrow$ 11    |
| ATGTGATGTCTCT <u></u>  | 3         |                        |

On obtient donc la suite de code : [0; 3; 2; 5; 4; 6; 1; 1; 3]

2. Quel est le taux de compression obtenu en supposant qu'un octet permet de représenter chaque code ?

12 sur 9

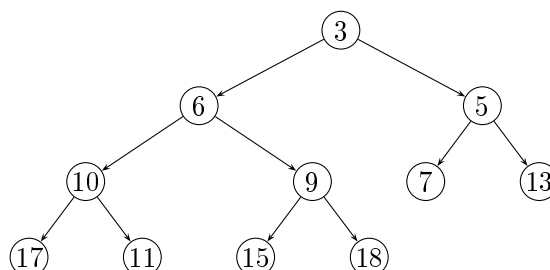
3. Décompresser le texte  $T$  codé par suite de codes [3; 1; 4; 6; 5; 2; 0; 2] sur ce même alphabet en expliquant comment est reconstruit le dictionnaire de décompression.

$T = \text{TCTCTCTCTGAG}$

4. Rappeler rapidement le principe de l'algorithme de compression de Huffman et compresser le texte  $T$  à l'aide de cet algorithme.

## □ Exercice : type B

- Rappeler la définition d'un tas binaire (min)
- On suppose qu'un tas est représenté par un tableau  $t = (t_0, \dots, t_{n-1})$ . Lorsqu'ils existent quels sont les indices des fils de  $t_i$  ?
- Quel est l'indice (lorsqu'il existe) du père de  $t_i$  ?
- Vérifier que l'arbre binaire suivant possède bien la structure de tas et donner sa représentation sous forme de tableau :



5. Rappeler le principe d'insertion d'un nouvel élément dans un tas binaire puis détailler les étapes de l'insertion de 4 dans le tas représenté à la question 4.
6. Rappeler le principe d'extraction du minimum d'un tas binaire, puis détailler les étapes de l'extraction du minimum du tas représenté à la question 4.
7. Dans la suite, on utilisera la structure de données suivante permettant de représenter un tas en OCaml :

```
1 type 'a heap = {mutable size : int; data : 'a array};;
```

Quel est le rôle du champ `size` ? Pourquoi est-il déclaré en `mutable` ?

8. Ecrire une fonction qui renvoie un tas de taille maximale 1000 et qui contient des valeurs entières.
9. Ecrire la fonction d'insertion d'un élément dans un tas binaire contenant des entiers. On renvoie `true` si l'insertion est possible (c'est-à-dire que le tas n'est pas plein) et `false` sinon. La signature de cette fonction est donc `insertion: 'a -> 'a heap -> bool`